

Gabriel Wolff
Vitor Camillo
João Victor Alvarez
Lucas Ashikaga
Wellington Fonseca

Arquitetura de Servidores: Infográfico Interativo
como projeto didático para Fundamentos de Estruturas de
Computadores

Curitiba
Junho de 2026

Gabriel Wolff
Vitor Camillo
João Victor Alvarez
Lucas Ashikaga
Wellington Fonseca

**Arquitetura de Servidores: Infográfico Interativo
como projeto didático para Fundamentos de Estruturas de
Computadores**

Trabalho final apresentado à disciplina de Fundamentos de Estruturas de Computadores, ministrada pelo Prof. Rubem Koide, Universidade Positivo, semestre 2026.1, como requisito parcial de avaliação.

Universidade Positivo
Escola Politécnica
Disciplina de Fundamentos de Estruturas de Computadores

Orientador: Prof. Rubem Koide

Curitiba
Junho de 2026

RESUMO

Este trabalho apresenta o desenvolvimento de um **infográfico interativo** sobre arquitetura de servidores como projeto didático da disciplina de Fundamentos de Estruturas de Computadores. O artefato, escrito em HTML5, CSS3 e JavaScript ES6+ sem dependências externas, organiza-se em seis módulos navegáveis que cobrem: (i) comparação Desktop *vs.* Servidor com tabelas de barramentos e interconexão multi-socket; (ii) RAID com simulador interativo de falhas, calculadora de IOPS e aplicação da Lei de Amdahl; (iii) memória ECC e código de Hamming SECDED; (iv) hierarquia de memória, cache associativa, NUMA e protocolo MESI; (v) virtualização, anéis de proteção, contêineres *vs.* VMs e escalonadores; e (vi) síntese integrando todas as camadas do *stack* de um servidor de produção. O projeto materializa a teoria das aulas 6 a 12 do semestre em demonstrações executáveis, com referencial bibliográfico fundamentado em Stallings, Patterson & Hennessy, Tanenbaum, Silberschatz e Russinovich.

Palavras-chave: arquitetura de computadores; servidores; RAID; memória ECC; NUMA; virtualização; hipervisor; infográfico interativo.

SUMÁRIO

1	INTRODUÇÃO	5
1.1	Justificativa	5
1.2	Objetivos	6
1.2.1	Objetivo geral	6
1.2.2	Objetivos específicos	6
1.3	Estrutura do trabalho	6
2	REFERENCIAL TEÓRICO	7
2.1	Desktop <i>versus</i> Servidor	7
2.2	RAID — Redundant Array of Independent Disks	8
2.2.1	Aplicação da Lei de Amdahl	8
2.3	Memória ECC e o código de Hamming	8
2.4	Hierarquia de memória, cache e NUMA	9
2.4.1	Organização da cache	9
2.4.2	NUMA — Non-Uniform Memory Access	10
2.4.3	Coerência: MESI, MESIF, MOESI	10
2.4.4	Paginação, TLB e <i>huge pages</i>	10
2.5	Virtualização e hipervisores	10
2.5.1	Tipo 1 vs. Tipo 2	11
2.5.2	Contêineres: virtualização ao nível do SO	11
2.5.3	Escalonadores	11
2.6	Síntese: <i>stack</i> completo do servidor	11
3	MATERIAIS E MÉTODOS	12
3.1	Hardware utilizado no desenvolvimento	12
3.2	Ferramentas e ambiente de desenvolvimento	12
3.2.1	Editor de código	12
3.2.2	Assistentes de IA generativa	13
3.2.3	Versionamento e hospedagem	13
3.2.4	Stack tecnológico do artefato	13
3.3	Metodologia de desenvolvimento	14
3.4	Divisão de responsabilidades na equipe	14
3.5	Critérios de avaliação e entrega	15
4	DESENVOLVIMENTO E RESULTADOS	16
4.1	Arquitetura geral do infográfico	16

4.2	Módulo 1 — Desktop vs. Servidor	17
4.2.1	Tabela comparativa de interconexões	18
4.3	Módulo 2 — RAID Interativo	18
4.3.1	Lógica do simulador de falhas	18
4.3.2	Calculadora de IOPS	20
4.4	Módulo 3 — Memória ECC e Hamming secded	20
4.4.1	Cálculo do síndrome	21
4.5	Módulo 4 — Hierarquia, Cache e NUMA	21
4.5.1	Simulador de acesso à cache	23
4.6	Módulo 5 — Virtualização	23
4.7	Módulo 6 — <i>Stack</i> Completo	23
4.7.1	Modelo de dados das camadas	24
4.8	Módulo de Referências	26
4.9	Resultado consolidado	26
5	CONCLUSÃO	28
5.1	Aderência aos objetivos	28
5.2	Limitações	29
5.3	Trabalhos futuros	29
5.4	Considerações finais	29
	REFERÊNCIAS	31
	ANEXOS	33
	ANEXO A – CÓDIGO-FONTE DO INFOGRÁFICO	34
A.1	Repositório completo	34
A.2	Estrutura do arquivo único	34
A.3	Função de navegação por sub-abas	35
A.4	Cálculo do simulador RAID	35
A.5	Aplicação da Lei de Amdahl	36
A.6	Simulador de <i>cache hit/miss</i>	37
A.7	Navegação para referência citada	37
A.8	Como executar localmente	37

1 INTRODUÇÃO

A disciplina de **Fundamentos de Estruturas de Computadores**, ministrada pelo Prof. Rubem Koide na Universidade Positivo, percorre um arco que vai dos conjuntos numéricos e portas lógicas (aulas 2–4) até processadores CISC/RISC e sistemas operacionais (aulas 10–12), passando pela arquitetura interna do computador e seus componentes físicos (aulas 6–9). Em paralelo a essa progressão teórica, o curso demanda dos alunos um projeto que materialize, em forma de artefato didático, um recorte significativo desse conteúdo.

Este trabalho apresenta o desenvolvimento de um **infográfico interativo sobre arquitetura de servidores**, escrito em HTML5, CSS3 e JavaScript ES6+ sem qualquer dependência externa, executável diretamente em navegador moderno. O recorte “servidores” foi escolhido por concentrar, num único objeto de estudo, todos os subsistemas que a disciplina aborda: do barramento e da hierarquia de memória até o escalonador de processos e a virtualização — portanto, exhibe a teoria das aulas em ação.

1.1 JUSTIFICATIVA

A primeira versão do projeto, entregue em abril de 2026, recebeu como crítica do orientador a *baixa profundidade de conteúdo*: o artefato apresentava analogias e curiosidades, mas pouca substância técnica que se conectasse de fato com o material das aulas. Este relatório descreve a **segunda iteração**, na qual o infográfico foi reestruturado para incorporar:

- tabelas comparativas com números reais de largura de banda, latência e custo (PCIe, UPI, DDR5, NVMe);
- fórmulas explícitas (Lei de Amdahl, cálculo de paridade em RAID 5, síndrome de Hamming SECDED);
- *specs* concretas de processadores citados em aula (Intel Core i7-5960X, Xeon Scalable, EPYC);
- simuladores executáveis (acesso à cache hit/miss, calculadora de IOPS, escalonador rubro-negro CFS); e
- bibliografia formal ABNT NBR 6023 com 17 referências, acessíveis diretamente da página via citações clicáveis.

1.2 OBJETIVOS

1.2.1 Objetivo geral

Desenvolver um **infográfico interativo** que explique, de modo visual e exploratório, os principais subsistemas que diferenciam um servidor de um computador pessoal, conectando explicitamente cada conceito ao conteúdo teórico das aulas 6–12 da disciplina.

1.2.2 Objetivos específicos

1. Comparar Desktop *vs.* Servidor em quatro dimensões — alimentação, interconexão CPU–CPU, barramentos de E/S e gerenciamento remoto — usando dados quantitativos.
2. Demonstrar interativamente o comportamento de quatro níveis RAID (0, 1, 5, 10) sob falha de disco e permitir o cálculo de capacidade útil e IOPS sob variação de parâmetros.
3. Visualizar o código de Hamming SECDED usado em memória ECC, expondo a posição dos bits de paridade e o cálculo do síndrome.
4. Apresentar a hierarquia de memória com latências em escala humana e simular a probabilidade de *hit* em cada nível de cache.
5. Explicar virtualização de Tipo 1 *vs.* Tipo 2, anéis de proteção, contêineres *vs.* máquinas virtuais e o escalonador CFS do Linux.
6. Integrar todas as camadas anteriores numa visão única do *stack* completo de um servidor moderno.

1.3 ESTRUTURA DO TRABALHO

O Capítulo 2 estabelece o referencial teórico, agrupado pelos seis subsistemas tratados no infográfico. O Capítulo 3 descreve materiais, métodos e o ambiente de desenvolvimento, incluindo as ferramentas de IA assistiva empregadas. O Capítulo 4 apresenta o infográfico módulo a módulo, com tabelas, figuras e descrição das simulações implementadas. O Capítulo 5 sintetiza os resultados, limitações e trabalhos futuros. O Anexo A traz o *link* do código-fonte completo no GitHub e trechos críticos do HTML/JavaScript desenvolvido.

2 REFERENCIAL TEÓRICO

Este capítulo apresenta o referencial conceitual sobre o qual o infográfico se apoia. As subseções a seguir espelham a estrutura modular do artefato e estão ancoradas tanto nos livros-texto da disciplina quanto em artigos seminais e material didático do Prof. Koide ([KOIDE, 2026](#)).

2.1 DESKTOP *VERSUS* SERVIDOR

Embora compartilhem o modelo de von Neumann ([STALLINGS, 2017](#)), desktop e servidor divergem em praticamente todos os subsistemas físicos. Servidores exigem altíssima disponibilidade — tipicamente SLA de 99,99 % (4,38 minutos de *downtime* mensais) ou 99,999 % (*five-nines*, 26 segundos mensais) — e operação ininterrupta 24/7. Para isso adotam:

- **Fontes redundantes *hot-plug*** (configuração N+1 ou 2N), onde a queima de uma unidade não interrompe a operação;
- **Arquitetura multi-socket** (2, 4 ou 8 processadores numa mesma placa), com interconexão coerente ponto-a-ponto — QPI/UPI na Intel ([INTEL CORPORATION, 2009](#)), HyperTransport/Infinity Fabric na AMD;
- **Memória ecc registrada** (RDIMM ou LRDIMM), capaz de detectar e corrigir erros de bit em tempo real;
- **Form factor padronizado em rack** EIA-310 (unidade básica 1U = 44,45 mm), permitindo densidade e padronização em *datacenters*;
- **Gerenciamento *out-of-band*** por microcontroladores dedicados como iDRAC (Dell), iLO (HPE) ou IPMI genérico, com endereço IP próprio e independente do sistema operacional principal;
- **Redes de alto desempenho** (10 GBE, 25 GBE, 100 GBE ou INFINIBAND) com *bonding* LACP para tolerância a falhas e agregação de banda.

O surgimento de cargas de trabalho de inteligência artificial generativa redesenhou o conceito de “servidor” nos últimos cinco anos. A planta passou a ser dominada por aceleradores — GPUS, TPUS, NPUS — com interconexões dedicadas (NVLink, CXL) que ultrapassam o PCIe em ordens de grandeza ([NVIDIA CORPORATION, 2023](#)).

2.2 RAID — REDUNDANT ARRAY OF INDEPENDENT DISKS

A proposta original de Patterson, Gibson e Katz ([PATTERSON; GIBSON; KATZ, 1988](#)) em 1988 introduziu a ideia de agrupar múltiplos discos baratos para obter, ao mesmo tempo, desempenho e tolerância a falhas *barata*. Os principais níveis adotados na indústria são:

RAID 0 (Striping) divide blocos de dados sequencialmente entre N discos; capacidade útil $N \cdot D$, leitura e escrita paralelas, *zero* tolerância a falhas.

RAID 1 (Mirroring) mantém cópia idêntica em dois discos; capacidade útil D , sobrevive à perda de $N - 1$ unidades de um grupo espelhado.

RAID 5 (Paridade distribuída) espalha um bloco de paridade $P = A \oplus B \oplus C$ entre os discos; capacidade útil $(N - 1) \cdot D$, sobrevive à perda de *um* disco, impõe penalidade de escrita $4\times$ (*read-modify-write*).

RAID 6 usa dois blocos de paridade independentes (P e Q); tolera *dois* discos quebrados, com penalidade $6\times$.

RAID 10 combina espelhamento + *striping*; alta performance e tolerância, custa metade da capacidade bruta.

2.2.1 Aplicação da Lei de Amdahl

A Lei de Amdahl ([AMDAHL, 1967](#)) delimita o ganho máximo de qualquer otimização paralela. Sendo P a fração paralelizável do trabalho e N o número de unidades de execução:

$$\text{Speedup}(N) = \frac{1}{(1 - P) + \frac{P}{N}} \quad (2.1)$$

Quando $N \rightarrow \infty$, $\text{Speedup} \rightarrow 1/(1 - P)$. No exemplo do servidor *www* discutido pelo Prof. Koide na Aula 9 (disco 10 000 RPM, *seek* médio de 8 MS, páginas de 8 KB), aproximadamente 99,3% do tempo de resposta é latência rotacional — portanto serial — e apenas 0,7% é transferência paralelizável. Substituindo na Equação 2.1 com $P = 0,007$, o *speedup* máximo teórico é $1/(1 - 0,007) \approx 1,007\times$. Conclusão pedagógica: paralelizar discos para esse perfil não ajuda; o vetor de otimização é *trocar a parte serial* — substituir o HDD por SSD.

2.3 MEMÓRIA ECC E O CÓDIGO DE HAMMING

Em servidor 24/7, *soft errors* de memória são estatisticamente certos. Partículas subatômicas (nêutrons cósmicos secundários e partículas α provenientes de contaminação

de solda) podem inverter um bit de DRAM sem aviso. O estudo de campo de Schroeder, Pinheiro e Weber ([SCHROEDER; PINHEIRO; WEBER, 2009](#)), realizado durante 2,5 anos na frota do Google, registrou taxas de 25 000 a 70 000 erros corrigíveis por DIMM ao ano e mostrou que aproximadamente 8 % dos DIMMs exibem ao menos um erro por ano — contrariando a hipótese anterior de erros raros ou esporádicos.

A correção em tempo real é feita por um código corretor proposto originalmente por Hamming ([HAMMING, 1950](#)) em 1950. A variante SECDED (*Single Error Correction, Double Error Detection*), padrão em DIMM ECC registrado, anexa quatro bits de paridade nas posições 2^k ($k = 0, 1, 2, 3$) e um bit adicional global. Cada bit de paridade cobre um subconjunto de posições determinado por seu bit correspondente em binário; na leitura, o controlador recalcula as paridades e obtém um *síndrome* cuja representação decimal é exatamente a posição do bit corrompido. O hardware inverte o bit indicado e entrega o dado correto à CPU sem qualquer participação do sistema operacional.

A norma JEDEC JESD79-5 ([JEDEC SOLID STATE TECHNOLOGY ASSOCIATION, 2024](#)) tornou obrigatória a presença de ECC *on-die* na DDR5, mas é importante notar: trata-se de proteção *interna* à célula, não cobrindo o *datapath* DIMM → CPU. Somente RDIMM/LRDIMM com ECC de fato protegem todo o canal.

2.4 HIERARQUIA DE MEMÓRIA, CACHE E NUMA

Patterson e Hennessy ([PATTERSON; HENNESSY, 2017](#)) formalizam o princípio: nenhuma tecnologia de memória é simultaneamente rápida, grande e barata. A solução é dispor camadas em pirâmide: registradores (sub-nanossegundo) → cache L1/L2/L3 (nanossegundos) → RAM (dezenas de nanossegundos) → SSD (microssegundos) → HDD (milissegundos). A eficiência da pirâmide depende de duas formas de localidade: **temporal** (dados acessados recentemente serão acessados de novo) e **espacial** (vizinhos de um dado acessado serão acessados em breve).

2.4.1 Organização da cache

Caches modernos são *set-associative*: o endereço seleciona um *set* e a linha pode ocupar qualquer um dos k *ways*. Um Intel Core i7-5960X, processador de referência citado no material de aula ([KOIDE, 2026](#)), organiza-se assim:

Nível	Tamanho	Associatividade	Linha	Latência
L1 (dados/instruções)	32 KB	8-way	64 B	4 ciclos
L2	256 KB	8-way	64 B	12 ciclos
L3 (<i>LLC</i>)	20 MB	20-way	64 B	38 ciclos

A **linha de cache de 64 bytes** é a unidade indivisível: ler um byte traz consigo 63 vizinhos. L1 é privada de cada *core*, L3 é compartilhada por todos os *cores* do mesmo *socket*.

2.4.2 NUMA — Non-Uniform Memory Access

Em servidor multi-socket, cada CPU possui seus pentes próprios de RAM. O acesso ao banco local é rápido (≈ 80 ns), mas o acesso à RAM do socket vizinho passa pelo link UPI/Infinity Fabric e custa cerca de 140 ns (STALLINGS, 2017). Essa não-uniformidade obriga o escalonador a manter cada *thread* próxima da memória que ela toca — conceito de *NUMA affinity* suportado por `numactl` (Linux) e funções equivalentes da API Windows.

2.4.3 Coerência: MESI, MESIF, MOESI

Quando múltiplos *cores* têm cópias da mesma linha de cache, é preciso garantir consistência. O protocolo MESI define quatro estados por linha: *Modified*, *Exclusive*, *Shared*, *Invalid*. A Intel acrescenta *Forward* (MESIF) e a AMD acrescenta *Owned* (MOESI). Toda escrita em linha *Shared* dispara invalidações nos outros *cores*; essa operação cruza o link de coerência e custa centenas de ciclos — o conhecido *false sharing*.

2.4.4 Paginação, TLB e *huge pages*

A MMU traduz endereços virtuais para físicos via tabelas de página percorridas em 4 níveis na arquitetura x86-64. A TLB (*Translation Lookaside Buffer*) cacheia as traduções recentes; quando há *miss*, o *page walk* custa até quatro acessos à RAM (SILBERSCHATZ; GALVIN; GAGNE, 2015). Para reduzir pressão na TLB, sistemas modernos suportam páginas de 2 MB (*huge pages*) e 1 GB (*gigantic pages*), cada entrada cobrindo $512\times$ ou $262\,144\times$ mais memória.

2.5 VIRTUALIZAÇÃO E HIPERVISORES

Popek e Goldberg (POPEK; GOLDBERG, 1974) estabeleceram, em 1974, três condições para que uma arquitetura seja virtualizável: **equivalência** (a VM deve se comportar como hardware nativo), **eficiência** (a maior parte das instruções deve executar sem intervenção do hipervisor) e **controle de recursos** (o hipervisor mantém posse exclusiva dos recursos físicos). Esses critérios não eram naturalmente satisfeitos pela arquitetura x86 original; somente em 2005 (Intel VT-X) e 2006 (AMD-V) as extensões de hardware introduziram o estado VMX (informalmente chamado de Ring -1) que coloca o hipervisor abaixo do Ring 0 (RUSSINOVICH; SOLOMON; IONESCU, 2017).

2.5.1 Tipo 1 vs. Tipo 2

- **Tipo 1 (*bare metal*)**: o hipervisor é o próprio sistema operacional raiz, instalado diretamente sobre o hardware. Exemplos: VMware ESXi, KVM (kernel Linux + QEMU), Hyper-V Server, Proxmox VE, Citrix XenServer. Performance próxima da nativa.
- **Tipo 2 (*hosted*)**: o hipervisor é um programa rodando sobre um sistema operacional convencional. Exemplos: VirtualBox (citado pelo Prof. Koide na Aula 12), VMware Workstation, Parallels Desktop. Performance reduzida pelo *overhead* do SO hospedeiro.

O Windows Subsystem for Linux 2 (WSL 2) ilustra o caso limítrofe: é um *kernel* Linux verdadeiro rodando numa VM *utility* leve gerenciada pelo Hyper-V, com *boot* quase instantâneo e integração transparente com o filesystem do Windows via 9P.

2.5.2 Contêineres: virtualização ao nível do SO

Diferentemente da VM, o contêiner compartilha o *kernel* do *host* — isolamento é obtido por *namespaces* (PID, mount, network, IPC, UTS, user) e *cgroups* do Linux (BOVET; CESATI, 2005). O resultado: *boot* em ≈ 100 MS, *footprint* de MBs, densidade acima de 1 000 contêineres por *host*. Orquestradores como Kubernetes escalam essa unidade para clusters de milhares de nós.

2.5.3 Escalonadores

O *Completely Fair Scheduler* (CFS) do Linux mantém todas as `task_struct` numa árvore rubro-negra ordenada por `vruntime` — tempo virtual de execução ponderado pela prioridade *nice*. A cada decisão, escolhe-se a tarefa mais à esquerda; complexidade $O(\log n)$ (BOVET; CESATI, 2005). O Windows usa escalonador *multilevel feedback queue* baseado em quantum e prioridades 0–31, com a maior parte da política implementada no `ntoskrnl.exe` (RUSSINOVICH; SOLOMON; IONESCU, 2017).

2.6 SÍNTESE: STACK COMPLETO DO SERVIDOR

Os cinco subsistemas anteriores não são compartimentos isolados. Numa requisição HTTP simples a um *e-commerce*, todos eles participam: o *load balancer* entrega o pacote pela rede (camada 2.1); o *ingress controller* no Kubernetes despacha para um *pod* (virtualização, 2.5); o processo executa em vCPUs mapeados em *cores* físicos com afinidade NUMA (2.4); buffers são alocados em RAM ECC (2.3); páginas são lidas de um SSD integrado a RAID 10 (2.2); e a resposta retorna percorrendo a mesma cadeia em sentido inverso.

A última seção do infográfico, descrita no Capítulo 4, materializa essa visão integradora.

3 MATERIAIS E MÉTODOS

Este capítulo descreve o ambiente físico, as ferramentas de software, a abordagem metodológica e a divisão de responsabilidades adotadas durante o desenvolvimento do infográfico.

3.1 HARDWARE UTILIZADO NO DESENVOLVIMENTO

O artefato foi desenvolvido em um **notebook Acer Aspire A315-42** com configuração de uso típico de aluno de graduação, conforme detalhado na Tabela 1.

Tabela 1 – Especificações do equipamento de desenvolvimento

Componente	Especificação
Processador	AMD Ryzen 5 3500U, 4 <i>cores</i> físicos / 8 <i>threads</i> , 2,1 GHz base
Arquitetura	x86_64, AMD Zen+, fabricação 12 nm
Memória RAM	9,6 GiB DDR4 2400 MHz (não-ECC, conforme classe consumidor)
Armazenamento	SSD NVMe M.2, 512 GB
Sistema operacional	EndeavourOS (base Arch Linux), <i>kernel</i> 7.0.9-zen1
Navegador alvo	Chromium 134+, Mozilla Firefox 130+

Por se tratar de aplicação web client-side, o infográfico foi testado também em equipamentos diversos dos demais integrantes da equipe, com sistemas operacionais Windows 11 e macOS 14, confirmando compatibilidade transversal nos navegadores baseados em Chromium e no Mozilla Firefox.

3.2 FERRAMENTAS E AMBIENTE DE DESENVOLVIMENTO

3.2.1 Editor de código

O desenvolvimento foi conduzido no **Visual Studio Code** ([MICROSOFT CORPORATION, 2026](#)), distribuído pela Microsoft sob licença MIT. As extensões utilizadas restringiram-se ao conjunto mínimo necessário: *Prettier* para formatação consistente, *Live Server* para auto-reload durante o desenvolvimento e *GitLens* para visualização do histórico de commits.

3.2.2 Assistentes de IA generativa

A produção utilizou duas ferramentas de IA, declaradas conforme a boa prática acadêmica atual de transparência sobre uso de modelos generativos:

Google Gemini — modo Canvas ([GOOGLE LLC, 2026](#)). Empregado na fase inicial de prototipagem, especialmente na geração de andaime visual em HTML/CSS para os módulos interativos (estrutura inicial do simulador RAID e das barras de recursos da virtualização).

Anthropic Claude Code ([ANTHROPIC, 2026](#)). Empregado na segunda iteração para aprofundamento técnico do conteúdo, refatoração da disposição em sub-abas, escrita das tabelas comparativas, redação deste relatório e elaboração do roteiro de defesa. A interação se deu via terminal sobre o diretório do projeto, com revisão humana de cada commit.

Ambas as ferramentas atuaram como copiloto: o discernimento técnico, a verificação dos números citados e a aderência ao material da disciplina foram conduzidos pelos integrantes da equipe.

3.2.3 Versionamento e hospedagem

O código-fonte está versionado em Git e hospedado no GitHub no repositório <https://github.com/glkwolff/Projeto-Estruturas-de-computadores>.

3.2.4 Stack tecnológico do artefato

O infográfico final é *auto-contido*: um único arquivo `projeto.html` (~2.500 linhas) contendo estrutura, estilos e lógica. Não há *build step*, transpilação ou dependência de pacotes externos. A Tabela 2 resume.

Tabela 2 – Tecnologias usadas no artefato final

Tecnologia	Uso
HTML5	Estrutura semântica em <code>section/ article, table</code>
CSS3	<i>Custom properties</i> , <code>grid</code> , <code>flex</code> , <code>clip-path</code> , transições, <i>media queries</i>
JavaScript ES6+	Manipulação de DOM, lógica de estado em variáveis-módulo, <code>setTimeout</code> para animações, <code>const/let</code>
SVG inline	(Não utilizado — toda visualização é feita por <code>divs</code> estilizadas, para reduzir dependência)

A escolha de *vanilla* JavaScript — sem frameworks como React ou Vue — foi deliberada: o objetivo pedagógico é exibir conceitos de arquitetura, não exercitar engenharia de *front-end*. Manter o projeto em arquivo único também simplifica entrega, avaliação e execução: basta abrir o HTML no navegador.

3.3 METODOLOGIA DE DESENVOLVIMENTO

O projeto seguiu duas iterações sucessivas:

1. **Iteração 1 (abril/2026).** Versão inicial entregue como parcial, focada em estabelecer o esqueleto visual e os cinco módulos básicos com analogias e curiosidades. Avaliada como insuficientemente profunda pelo orientador.
2. **Iteração 2 (junho/2026).** Refatoração com foco em *profundidade conteudística*: incorporação de tabelas quantitativas, fórmulas explícitas, simuladores adicionais (calculadora RAID, simulador de cache hit/miss, escalonador rubro-negro), reorganização de cada módulo em sub-abas para reduzir densidade visual e adição de um sexto módulo (*stack* completo) integrador.

A metodologia em cada módulo seguiu o ciclo:

1. leitura do material das aulas correspondente (aulas 6 a 12);
2. mapeamento dos conceitos para subseções navegáveis;
3. definição de uma interação central — “o que o aluno pode *fazer* aqui?” — em vez de ler texto passivo;
4. implementação progressiva, testando hot-reload no navegador;
5. revisão pela equipe e ajuste fino.

3.4 DIVISÃO DE RESPONSABILIDADES NA EQUIPE

A equipe é composta por cinco integrantes, com divisão temática para garantir profundidade individual em pelo menos um módulo (condição da nota de defesa, que é individual). A Tabela 3 resume.

A divisão temática não impediu colaboração em todos os módulos: cada integrante revisou e contribuiu para os demais, mas tem domínio aprofundado do tema que apresentará na defesa.

Tabela 3 – Distribuição de responsabilidades por integrante

Integrante	Módulo principal de apresentação
Gabriel Wolff	Módulo 1 — Desktop <i>vs.</i> Servidor
Vitor Camillo	Módulo 2 — RAID + Lei de Amdahl
João Victor Alvarez	Módulo 3 — Memória ECC + Hamming
Lucas Ashikaga	Módulo 4 — Hierarquia & NUMA
Wellington Fonseca	Módulo 5 — Virtualização
Equipe (coletivo)	Módulo 6 — <i>Stack</i> completo (síntese)

3.5 CRITÉRIOS DE AVALIAÇÃO E ENTREGA

Conforme orientação do professor, a avaliação é composta de duas notas independentes:

- **Apresentação de defesa** (individual, 2,5 pontos): duração máxima de 15 minutos, com cada integrante apresentando seu módulo principal.
- **Relatório escrito** (coletivo, 2,5 pontos): este documento, com estrutura ABNT.

A entrega final compreende um arquivo `.zip` contendo o relatório em PDF, a apresentação, o arquivo `projeto.html` do infográfico e o *link* para o repositório GitHub (uma vez que o código-fonte completo ultrapassa as 2 500 linhas, conforme preconizado pelo critério de entrega).

4 DESENVOLVIMENTO E RESULTADOS

Este capítulo descreve cada um dos seis módulos do infográfico, apresentando: (i) o conceito teórico que o módulo materializa, (ii) as sub-abas que dividem o conteúdo, (iii) as tabelas e dados incorporados e (iv) as simulações interativas implementadas em JavaScript. Reproduções das telas são apresentadas como Figuras 1–6 (capturadas em 3 de junho de 2026, navegador Firefox).

4.1 ARQUITETURA GERAL DO INFOGRÁFICO

O artefato organiza-se segundo uma navegação lateral com sete itens: seis módulos temáticos mais o índice de referências bibliográficas. Cada módulo, por sua vez, divide-se em quatro sub-abas que isolam um recorte conceitual — decisão de design tomada na segunda iteração para reduzir a densidade visual percebida na primeira versão.

A Listagem 4.1 mostra o esqueleto de navegação:

```

1 <nav>
2   <button onclick="showModule('mod1', this)">Desktop vs Servidor</button>
3   <button onclick="showModule('mod2', this)">RAID Interativo</button>
4   <button onclick="showModule('mod3', this)">Memória ECC</button>
5   <button onclick="showModule('mod4', this)">Hierarquia & NUMA</button>
6   <button onclick="showModule('mod5', this)">Virtualização</button>
7   <button onclick="showModule('mod6', this)">Stack Completo</button>
8   <button onclick="showModule('modRef', this)">Referências</button>
9 </nav>

```

Listing 4.1 – Esqueleto da navegação lateral

Internamente, cada módulo é uma `section` com `display:none` por padrão; a função `showModule()` ativa apenas a seção solicitada (Listagem 4.2).

```

1 function showModule(modId, element) {
2   document.querySelectorAll('.module').forEach(m =>
3     m.classList.remove('active'));
4   document.querySelectorAll('.nav-btn').forEach(b =>
5     b.classList.remove('active'));
6   document.getElementById(modId).classList.add('active');
7   if (element) element.classList.add('active');
8   window.scrollTo({ top: 0, behavior: 'smooth' });
9 }

```

Listing 4.2 – Função de navegação principal

4.2 MÓDULO 1 — DESKTOP VS. SERVIDOR

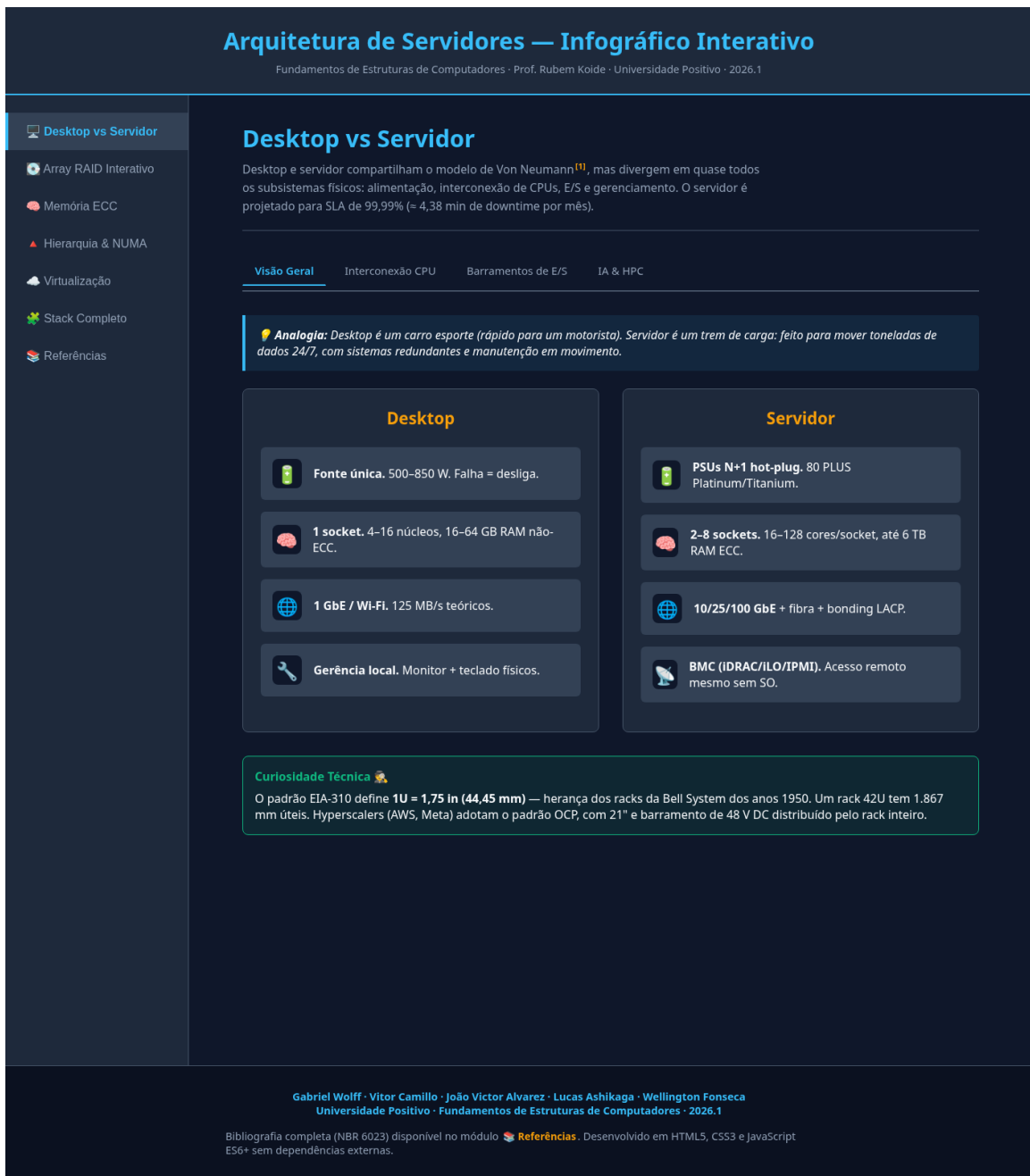


Figura 1 – Módulo 1 — Comparativo Desktop vs. Servidor.

O módulo apresenta quatro sub-abas:

Visão Geral grade comparativa lado-a-lado com 4 itens em cada coluna (alimentação, sockets, rede, gerência), mais analogia didática e curiosidade técnica sobre o padrão EIA-310.

Interconexão CPU diagrama *dual-socket* com link UPI explícito (41,6 GB/s, ~100 ns) e tabela comparativa de FSB, QPI, UPI, Infinity Fabric e NVLink.

Barramentos de E/S tabela quantitativa de PCIe 3.0/4.0/5.0, NVME, SATA, 1/10/100 GBE e INFINIBAND, com *throughput* em GB/s.

IA & HPC cards com servidores de inteligência artificial: NVIDIA Grace Hopper GH200, AMD Instinct MI300X, Google TPU v5p e o supercomputador *Frontier* (exaescala).

4.2.1 Tabela comparativa de interconexões

A Tabela 4 é a peça central da aba “Interconexão CPU”. Os números provêm do *white paper* da Intel ([INTEL CORPORATION, 2009](#)) e do material das aulas ([KOIDE, 2026](#)).

Tabela 4 – Tecnologias de interconexão multi-socket

Tecnologia	Fabricante	Banda	Coerência
FSB	Intel até 2008	10,6 GB/s	Snooping
QPI	Intel 2008–2017	25,6 GB/s	MESIF (4 camadas)
UPI	Intel Skylake+	41,6 GB/s	MESIF + diretório
Infinity Fabric	AMD	204 GB/s	MOESI
NVLink 4.0	NVIDIA	900 GB/s	GPU coherency

4.3 MÓDULO 2 — RAID INTERATIVO

Distribuído em quatro sub-abas:

Simulador reproduz a versão original do módulo — o aluno seleciona o nível RAID (0, 1, 5 ou 10), o sistema desenha os discos e calcula em tempo real o impacto de cliques sucessivos (falhas) sobre o *status* do *array*.

Níveis Comparados tabela com seis níveis (0, 1, 5, 6, 10, 50) detalhando capacidade útil, tolerância e penalidade de escrita, mais a fórmula explícita de paridade $P = A \oplus B \oplus C$.

Calculadora formulário onde o usuário escolhe nível, número de discos, capacidade individual e IOPS por disco; o sistema computa capacidade útil, eficiência de espaço, discos toleráveis e IOPS de leitura/escrita considerando a penalidade do nível selecionado.

Lei de Amdahl aplicação numérica do Capítulo 2, com simulador interativo (*slider* de fração paralelizável P e número de discos N) que recalcula o *speedup* efetivo e o teto teórico em tempo real.

4.3.1 Lógica do simulador de falhas

O estado do *array* é mantido num vetor JavaScript `raidState`; cada elemento descreve o ID do disco e se ele está em falha. A função `updateRaidStatus()` contém o `switch`

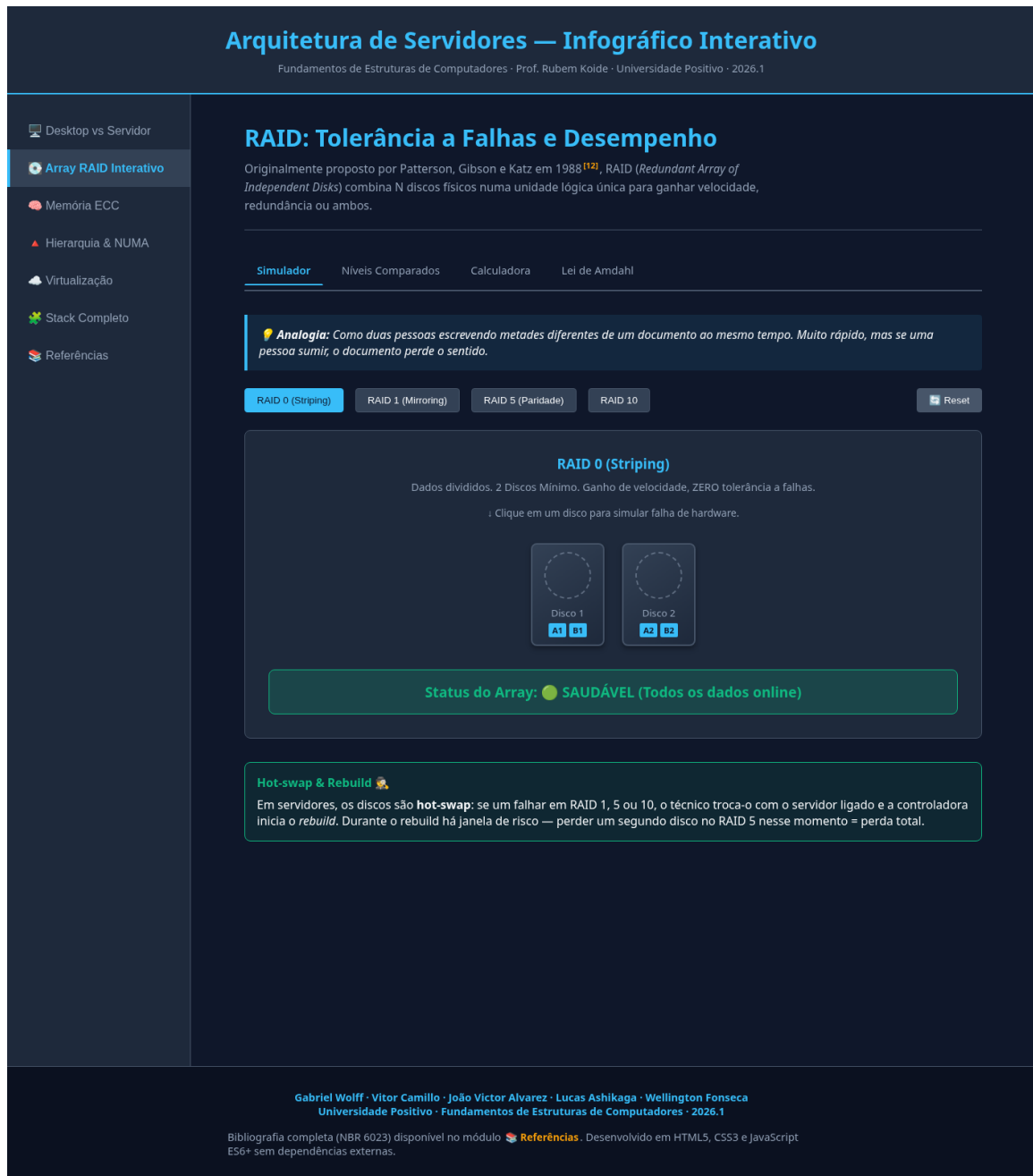


Figura 2 – Módulo 2 — RAID com simulador de falha.

que codifica o comportamento esperado de cada nível sob diferentes contagens de falhas (Listagem 4.3).

```

1  switch(currentRaid) {
2      case 0:
3          statusClass = 'status-fail';
4          statusStr   = 'PERDA TOTAL! RAID 0 não tolera falhas.';
5          break;
6      case 1:
7          if (failedCount === 1) { statusClass = 'status-warn';
8              statusStr = 'DEGRADADO! Operando só pelo espelho.'; }
9          else { statusClass = 'status-fail';

```

```

10     statusStr = 'FALHA CRITICA! Ambos espelhos perdidos.'; }
11     break;
12 case 5:
13     if (failedCount === 1) { statusClass = 'status-warn';
14         statusStr = 'DEGRADADO! Recalculando paridade.'; }
15     else { statusClass = 'status-fail';
16         statusStr = 'RAID 5 só tolera 1 falha.'; }
17     break;
18     /* ... RAID 10 omitido por brevidade ... */
19 }

```

Listing 4.3 – Trecho da lógica de status do RAID

4.3.2 Calculadora de IOPS

A fórmula adotada para a penalidade de escrita segue a literatura padrão de armazenamento empresarial:

$$\text{IOPS}_{\text{escrita}} = \frac{N \cdot \text{IOPS}_{\text{disco}}}{\text{penalty}} \quad \text{com penalty} \in \{1, 2, 4, 6, 2\} \quad (4.1)$$

para os níveis 0, 1, 5, 6, 10 respectivamente. Isso reproduz o conhecido “RAID 5 *write penalty*” de fator 4.

4.4 MÓDULO 3 — MEMÓRIA ECC E HAMMING SECDED

Quatro sub-abas:

Simulador herda a animação original de raio cósmico, comparando lado-a-lado RAM comum e RAM ECC.

Como funciona (Hamming) mostra os 12 bits da palavra Hamming(12,8): quatro paridades P_1, P_2, P_4, P_8 nas posições 2^k , oito bits de dado, e uma tabela com as posições cobertas por cada paridade.

Soft errors no mundo real expõe os números do estudo de Schroeder *et al.* (SCHROEDER; PINHEIRO; WEBER, 2009) e cita o caso da eleição de Schaerbeek (Bélgica, 2003) com *bit flip* no contador.

Modalidades de proteção tabela comparando paridade simples, SECDED, Chipkill/SDDC e DDR5 *on-die* ECC.



Figura 3 – Módulo 3 — Memória ECC: simulador de bit flip.

4.4.1 Cálculo do síndrome

Com 4 bits de paridade, a representação binária do *síndrome* recalculado na leitura tem 16 valores possíveis; 0 indica ausência de erro, e os demais indicam exatamente a posição do bit corrompido. Por exemplo: paridades $(s_3, s_2, s_1, s_0) = (0, 1, 0, 1) \Rightarrow$ posição decimal 5 $\Rightarrow$ corrupção em D_2 (o segundo bit de dado, na posição 5 do layout Hamming).

4.5 MÓDULO 4 — HIERARQUIA, CACHE E NUMA

Quatro sub-abas:

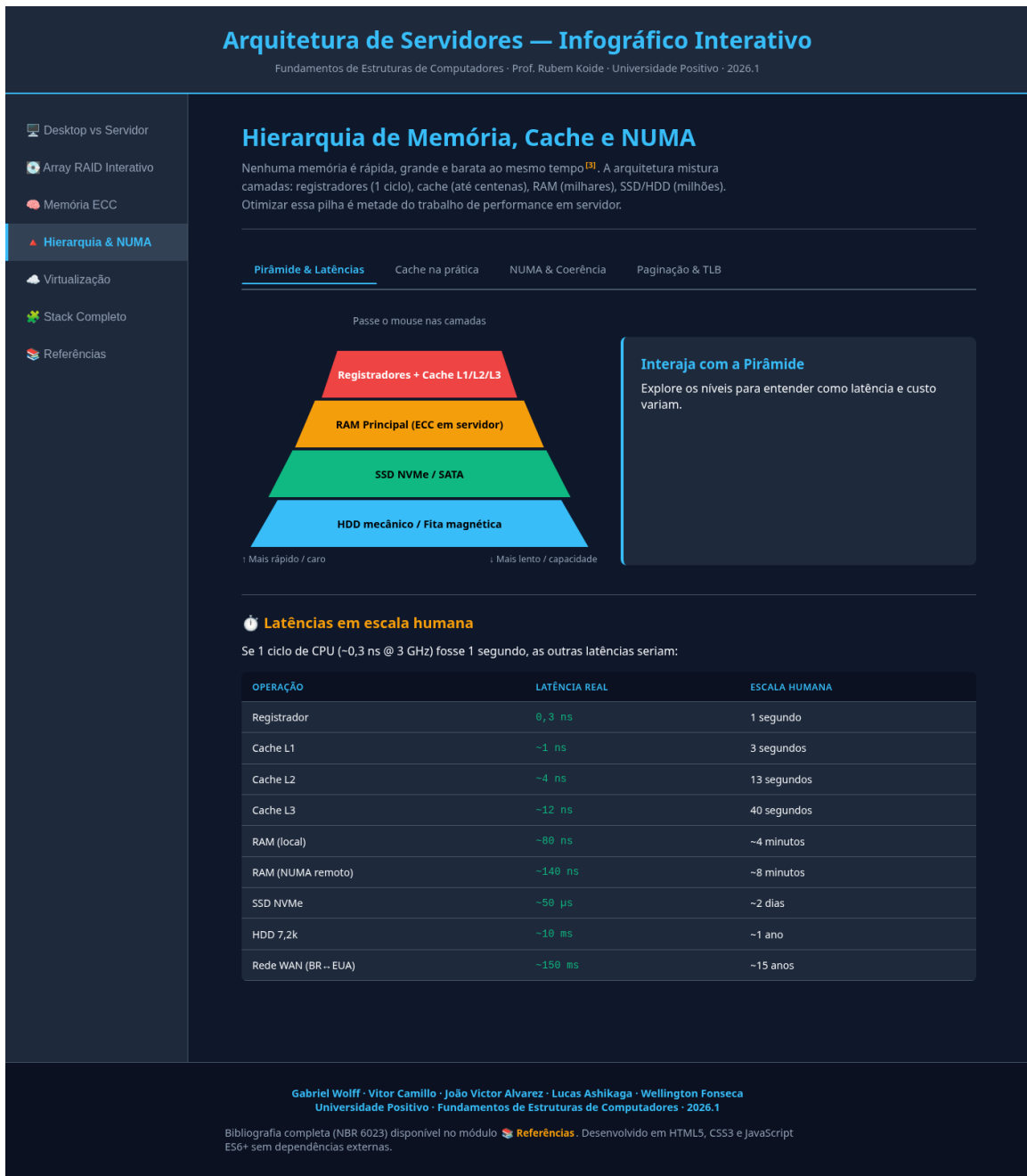


Figura 4 – Módulo 4 — Pirâmide de memória com latências em escala humana.

Pirâmide & Latências versão original da pirâmide interativa com hover, complementada por uma tabela que escalona as latências para a referência humana (se 1 ciclo de CPU equivalesse a 1 segundo, um acesso à RAM remota NUMA equivaleria a 8 minutos).

Cache na prática tabela com a hierarquia real de um Intel Core i7-5960X (Tabela 5) e simulador de *cache hit/miss* animado.

NUMA & Coerência diagrama *dual-socket* com latência local (80 ns) versus remota (140 ns), e tabela explicando os 4 estados do protocolo MESI.

Paginação & TLB páginas de 4 KB *vs.* *huge pages* de 2 MB, *page walk* em 4 níveis no x86-64, fórmula $t_{\text{acesso}} = t_{\text{tlb}} + (\text{taxa_miss} \cdot t_{\text{pagewalk}})$.

Tabela 5 – Cache do Intel Core i7-5960X (8 cores, Haswell-E)

Nível	Tamanho	Assoc.	Latência	Escopo
L1d	32 KB	8-way	4 ciclos	Por core
L1i	32 KB	8-way	4 ciclos	Por core
L2	256 KB	8-way	12 ciclos	Por core
L3	20 MB	20-way	38 ciclos	8 cores

4.5.1 Simulador de acesso à cache

Implementa probabilidades empíricas típicas: 92 % de *hit* em L1, 6 % em L2, 1,5 % em L3 e 0,5 % *miss* até a RAM. A cada clique no botão “Simular acesso”, a função `simulateCacheAccess()` sorteia o resultado e anima a travessia da pirâmide marcando *miss* em cinza e *hit* em verde, exibindo o custo em ciclos do nível atingido.

4.6 MÓDULO 5 — VIRTUALIZAÇÃO

Quatro sub-abas:

Simulador o alocador de VMs original, que limita a soma de vCPUs e RAM ao máximo físico (32 cores / 128 GB) e visualmente preenche barras de recurso, vermelha se acima de 80 %.

Tipo 1 *vs.* Tipo 2 tabela taxonômica + diagrama dos anéis de proteção x86, incluindo o VMX *root* (Ring −1) introduzido por VT-X/AMD-V, e curiosidade sobre WSL 2 como caso de virtualização leve.

VM *vs.* Contêiner tabela comparando granularidade, *boot time*, *footprint* e densidade, mais um trecho de código Linux mostrando criação manual de contêiner com `unshare` + `cgroups`.

Kernels & Escalonador comparação Windows NT (híbrido) *vs.* Linux (monolítico modular), com diagrama da árvore rubro-negra usada pelo CFS.

4.7 MÓDULO 6 — STACK COMPLETO

Este módulo **não tem sub-abas**: é uma síntese visual integrando todas as camadas anteriores num único diagrama vertical com 9 camadas clicáveis, do nível físico (PSU,

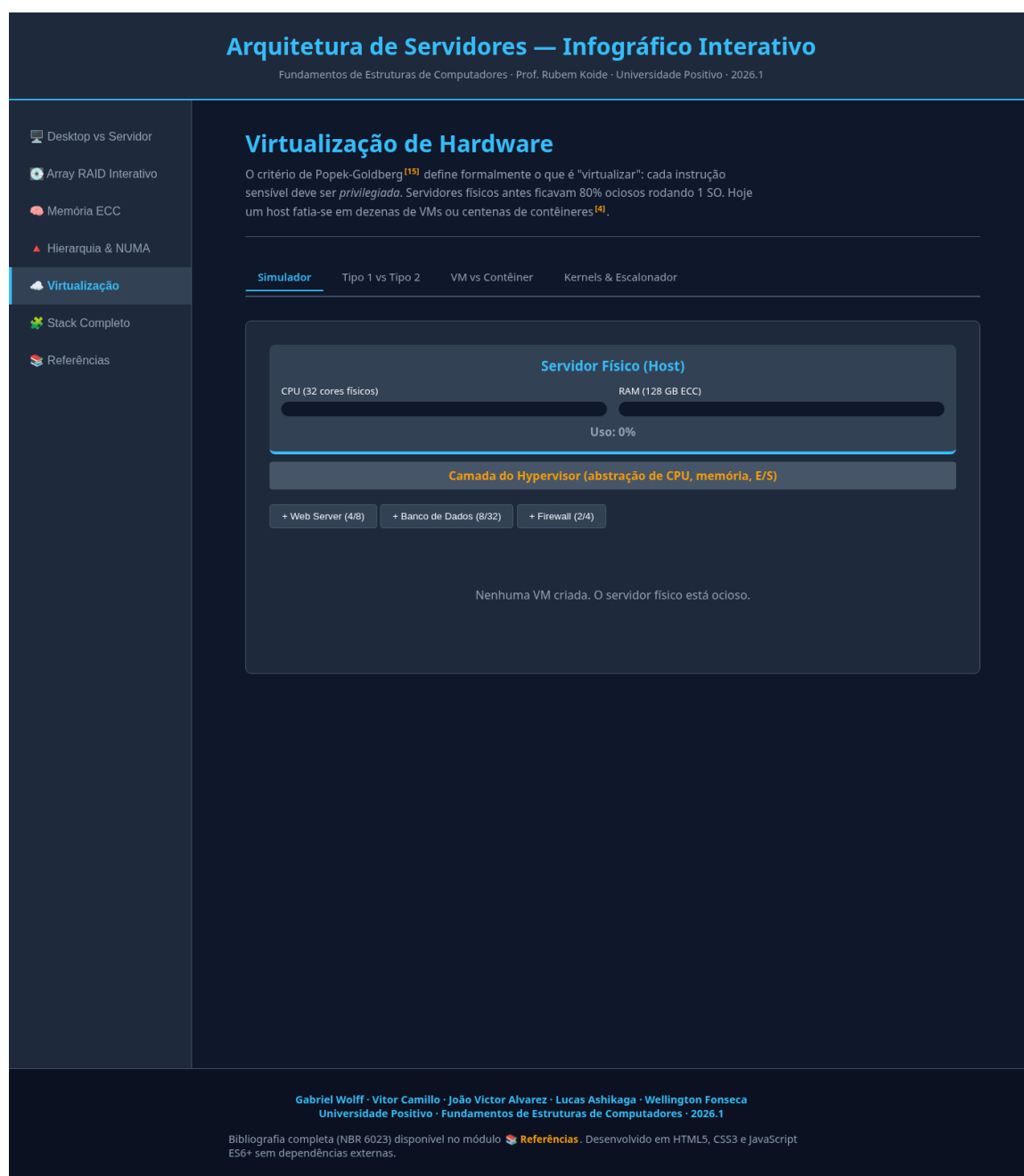


Figura 5 – Módulo 5 — Alocador de máquinas virtuais sobre o *host* físico.

BMC, rack) até a aplicação no topo (Nginx, PostgreSQL, microsserviços). Ao clicar em cada camada, o painel à direita exibe seu detalhamento técnico — com referências cruzadas aos módulos anteriores.

4.7.1 Modelo de dados das camadas

A informação de cada camada é mantida num objeto JavaScript `stackData`, conforme o trecho da Listagem 4.4.

```
1 const stackData = {
2   cpu: {
```

Arquitetura de Servidores — Infográfico Interativo

Fundamentos de Estruturas de Computadores · Prof. Rubem Koide · Universidade Positivo · 2026.1

- Desktop vs Servidor
- Array RAID Interativo
- Memória ECC
- Hierarquia & NUMA
- Virtualização
- Stack Completo
- Referências

Stack Completo do Servidor

Os cinco módulos anteriores não são pilares isolados — são *camadas*. Esta visão integra tudo: do gabinete físico até a aplicação web servindo o usuário final. Clique em cada camada para explorar.

- 1 Aplicação
Nginx, PostgreSQL, microserviços, jobs cron
- 2 Runtime / Contêiner
Docker, containerd, Kubernetes (pods)
- 3 Máquina Virtual
SO convidado: Ubuntu / RHEL / Win Server
- 4 Hypervisor (Tipo 1)
VMware ESXi, KVM, Proxmox VE, Hyper-V
- 5 CPU (Multi-Socket, NUMA)
Xeon / EPYC, UPI / Infinity Fabric, MESIF
- 6 Memória ECC (NUMA-aware)
DDR5 RDIMM, Hamming SECDED, on-die ECC
- 7 Storage (RAID + hot-swap)
NVMe U.3 + HBA, RAID 10/6, replica off-site
- 8 Rede (10/100 GbE)
NICs redundantes, fibra LC, LACP, SR-IOV
- 9 Infraestrutura física
Rack 42U, PSU N+1, BMC iDRAC/iLO, refrigeração

1 Aplicação

O topo da pilha — o que entrega valor ao usuário final. Tipicamente um conjunto de processos:

- Servidor web:** Nginx, Apache, IIS — responde HTTP.
- Aplicação:** Node, Java, .NET, Python — lógica de negócio.
- Banco de dados:** PostgreSQL, MySQL, MongoDB, Redis.
- Mensageria/cache:** Kafka, RabbitMQ, Memcached.

Para cada requisição, esse processo aciona *todas* as camadas abaixo: lê dados do disco (7), aloca buffers em RAM (6), executa em vCPUs (5), passa pelo hypervisor (4), entrega via rede (8). Otimizar performance significa entender essa cadeia inteira.

O efeito cascata de uma só requisição HTTP

Uma única requisição "GET /produto/123" num e-commerce moderno típico atravessa: load balancer (8) → ingress controller no pod K8s (2) → container app (2) → query SQL no DB pod (2) → driver NUMA-aware (6) → leitura de página em SSD NVMe via RAID controller (7) → ECC valida o byte que voltou (6) → resposta JSON volta toda a pilha em ~10 ms. **Cada camada custou microssegundos.**

Gabriel Wolff · Vitor Camillo · João Victor Alvarez · Lucas Ashikaga · Wellington Fonseca
Universidade Positivo · Fundamentos de Estruturas de Computadores · 2026.1

Bibliografia completa (NBR 6023) disponível no módulo [Referências](#). Desenvolvido em HTML5, CSS3 e JavaScript ES6+ sem dependências externas.

Figura 6 – Módulo 6 — *Stack* integrado do servidor de produção.

```

3      title: 'CPU (Multi-Socket, NUMA)',
4      body: '<p>Em servidor, 2-8 sockets físicos compartilham ' +
5            'a memória via UPI (Intel) ou Infinity Fabric (AMD)' +
6            ', com protocolo MESIF/MOESI para coerência.</p>' +
7            '<ul><li>Xeon Scalable / EPYC: 16-128 cores/socket</li>'
8
9            +
10           '<li>Hyperthreading: 2 threads lógicos por core físico</li>' +
11           '<li>NUMA-aware: scheduler mantém threads próximas à RAM'
12           '</li>' +
13           '</ul>'

```

```
11     },  
12     /* ... outras 8 camadas omitidas por brevidade ... */  
13 };
```

Listing 4.4 – Estrutura de dados das camadas do stack

A função `showStackDetail(key, el)` troca o painel da direita para o conteúdo da camada selecionada, mantendo o histórico de seleção via classe CSS `.active`.

4.8 MÓDULO DE REFERÊNCIAS

Um sétimo item de navegação — **Referências** — expõe a lista bibliográfica completa formatada conforme ABNT NBR 6023:2018. Cada citação inline nos módulos é um link ativo: ao clicar em [11], por exemplo, o usuário é transportado para a aba de referências e a entrada 11 é destacada em laranja por 2,5 segundos — comportamento implementado em `goToRef(n)` (Listagem 4.5).

```
1 function goToRef(n) {  
2     showModule('modRef', null);  
3     setTimeout(() => {  
4         const target = document.getElementById('ref-' + n);  
5         if (!target) return;  
6         target.scrollIntoView({ behavior: 'smooth', block: 'center' });  
7         target.classList.add('highlight');  
8         setTimeout(() => target.classList.remove('highlight'), 2500);  
9     }, 200);  
10 }
```

Listing 4.5 – Função de navegação para referência citada

4.9 RESULTADO CONSOLIDADO

O artefato final possui:

- 6 módulos temáticos + 1 índice de referências;
- 21 sub-abas no total (média de 3,5 por módulo);
- 17 referências bibliográficas ABNT clicáveis;
- 11 tabelas técnicas com dados quantitativos;
- 8 simulações interativas em JavaScript;
- aproximadamente 2 500 linhas em arquivo HTML único;

- zero dependências externas, executável *offline*.

O salto de profundidade entre as duas iterações pode ser quantificado de forma simples: a primeira versão do código tinha 1 060 linhas e 3 referências bibliográficas; a segunda chega a 2 554 linhas e 17 referências, com adição de todas as fórmulas, tabelas e simuladores listados acima.

5 CONCLUSÃO

O presente trabalho desenvolveu um infográfico interativo sobre arquitetura de servidores, materializando, em forma de artefato didático executável em navegador, o conteúdo das aulas 6 a 12 da disciplina de Fundamentos de Estruturas de Computadores.

O processo passou por duas iterações. A primeira versão estabeleceu o esqueleto visual e a interatividade básica, mas foi avaliada pelo orientador como insuficiente em profundidade técnica. A segunda iteração — a que este relatório descreve — refatorou cada módulo para incorporar tabelas comparativas com números reais (barramentos, latências, capacidades), fórmulas explícitas (Amdahl, paridade XOR, síndrome de Hamming), *specs* de processadores reais (Intel Core i7-5960X, Xeon Scalable, EPYC, NVIDIA Grace Hopper), bibliografia formal ABNT NBR 6023 acessível diretamente do infográfico via citações clicáveis, e um sexto módulo integrador (*stack* completo) que sintetiza os cinco anteriores numa única visão.

5.1 ADERÊNCIA AOS OBJETIVOS

Confrontados com os objetivos específicos enunciados na Seção 1.2, observamos:

- **Comparativo Desktop vs. Servidor:** atendido com a sub-aba “Visão Geral” e complementado com três aprofundamentos (Interconexão CPU, Barramentos de E/S, IA & HPC) ausentes na primeira versão.
- **Simulação de RAID e cálculo quantitativo:** atendido com o simulador original mais uma calculadora de IOPS e capacidade útil e a aplicação numérica da Lei de Amdahl ao exemplo do servidor `www`.
- **Visualização do código de Hamming:** atendido com a sub-aba “Como funciona”, que expõe os 12 bits da palavra Hamming(12,8) e a tabela de cobertura dos bits de paridade.
- **Hierarquia com latências em escala humana:** atendido com tabela conversora (ciclo de CPU $\rightarrow$ 1 segundo) e simulador de *cache hit/miss* animado.
- **Virtualização Tipo 1 vs. Tipo 2:** atendido com taxonomia, anéis de proteção x86 (Ring 0 a Ring -1) e comparação dedicada VM *vs.* contêiner.
- **Integração final:** atendido com o Módulo 6 (*stack* completo), inexistente na primeira versão.

5.2 LIMITAÇÕES

Algumas escolhas conscientes restringem o alcance do artefato:

1. Por se tratar de aplicação *client-side* sem *back-end*, nenhum dos simuladores executa medições reais de hardware; todos os números (IOPS, latência, *speedup*) são calculados via fórmulas determinísticas com parâmetros tabulares ou via amostragem de uma distribuição empírica.
2. A análise da Lei de Amdahl considera apenas o cenário ideal de paralelização homogênea entre discos, ignorando contenção de barramento, *cache pollution* e custo do controlador RAID.
3. A visualização do escalonador CFS é estática (árvore exemplificativa); não há simulador de inserção e remoção de *tasks* ao longo do tempo.
4. O suporte mobile é funcional (*media queries* em três *breakpoints*), mas tabelas extensas precisam de rolagem horizontal em telas estreitas.

5.3 TRABALHOS FUTUROS

- **Animar a coerência mesi/mesif:** implementar um simulador onde dois *cores* disputam uma linha de cache e o usuário acompanha as transições de estado.
- **Visualização do *page walk*:** animar a tradução de um endereço virtual nos 4 níveis da paginação x86-64, com e sem *TLB hit*.
- **Integração com benchmarks reais:** incorporar dados de *fio*, *stress-ng* e *lmbench* para ancorar os valores teóricos em medições reproduzíveis.
- **Tradução para o inglês:** o infográfico tem potencial além do escopo da disciplina — com tradução, poderia servir como recurso aberto (CC-BY) para outras turmas de Arquitetura de Computadores.
- **Acessibilidade:** auditar para conformidade WCAG 2.1 AA, garantindo navegação por teclado e leitor de tela em todos os simuladores.

5.4 CONSIDERAÇÕES FINAIS

O salto de profundidade entre as duas iterações — de 1 060 para 2 554 linhas de código, de 3 para 17 referências bibliográficas, de zero para 11 tabelas quantitativas e de 5 para 8 simuladores interativos — ilustra que a profundidade técnica de um artefato didático é fundamentalmente uma decisão *de conteúdo*, não de arquitetura de software.

Manter o projeto em um único arquivo HTML sem dependências facilitou justamente esse aprofundamento: toda a energia da segunda iteração pôde ser direcionada para a leitura crítica do material das aulas e para a tradução dos conceitos em visualizações executáveis, sem ser desperdiçada em configuração de *build tools* ou gerenciamento de pacotes.

O resultado é um artefato que, esperamos, faça jus à crítica original do orientador e demonstre, de forma navegável e auto-contida, por que um servidor é mais do que “um computador maior” — e quais subsistemas precisam estar bem compreendidos para projetá-lo, operá-lo ou estudá-lo.

REFERÊNCIAS

- AMDAHL, G. M. Validity of the single processor approach to achieving large scale computing capabilities. In: *AFIPS Conference Proceedings*. [S.l.: s.n.], 1967. v. 30, p. 483–485. 8
- ANTHROPIC. *Claude Code: assistente de IA para desenvolvimento de software*. San Francisco: Anthropic, 2026. Disponível em: <<https://claude.com/claude-code>>. Acesso em: 3 jun. 2026. 13
- BOVET, D. P.; CESATI, M. *Understanding the Linux Kernel*. 3. ed.. ed. Sebastopol: O'Reilly, 2005. 11
- GOOGLE LLC. *Gemini: assistente de IA generativa (modo Canvas)*. Mountain View: Google, 2026. Disponível em: <<https://gemini.google.com>>. Acesso em: 3 jun. 2026. 13
- HAMMING, R. W. Error detecting and error correcting codes. *Bell System Technical Journal*, v. 29, n. 2, p. 147–160, abr. 1950. 9
- INTEL CORPORATION. *An Introduction to the Intel® QuickPath Interconnect*. Santa Clara: Intel, 2009. White Paper, doc. 320412-001US. 7, 18
- JEDEC SOLID STATE TECHNOLOGY ASSOCIATION. *DDR5 SDRAM Standard — JESD79-5C*. Arlington: JEDEC, 2024. 9
- KOIDE, R. *Fundamentos de Estruturas de Computadores: material das aulas 5–12*. Curitiba: Universidade Positivo, 2026. Semestre letivo 2026.1. 7, 9, 18
- MICROSOFT CORPORATION. *Visual Studio Code: editor de código fonte*. Redmond: Microsoft, 2026. Disponível em: <<https://code.visualstudio.com>>. Acesso em: 3 jun. 2026. 12
- NVIDIA CORPORATION. *NVIDIA Grace Hopper Superchip Architecture Whitepaper*. Santa Clara: NVIDIA, 2023. Disponível em: <<https://resources.nvidia.com/en-us-grace-cpu/grace-hopper-superchip>>. Acesso em: 3 jun. 2026. 7
- PATTERSON, D. A.; GIBSON, G.; KATZ, R. H. A case for redundant arrays of inexpensive disks (RAID). In: *Proceedings of the 1988 ACM SIGMOD International Conference on Management of Data*. New York: ACM, 1988. p. 109–116. 8
- PATTERSON, D. A.; HENNESSY, J. L. *Organização e projeto de computadores: a interface hardware/software*. 5. ed.. ed. Rio de Janeiro: Elsevier, 2017. 9
- POPEK, G. J.; GOLDBERG, R. P. Formal requirements for virtualizable third generation architectures. *Communications of the ACM*, v. 17, n. 7, p. 412–421, jul. 1974. 10
- RUSSINOVICH, M. E.; SOLOMON, D. A.; IONESCU, A. *Windows Internals, Part 1*. 7. ed.. ed. Redmond: Microsoft Press, 2017. 10, 11
- SCHROEDER, B.; PINHEIRO, E.; WEBER, W.-D. DRAM errors in the wild: a large-scale field study. *ACM SIGMETRICS Performance Evaluation Review*, v. 37, n. 1, p. 193–204, 2009. 9, 20

SILBERSCHATZ, A.; GALVIN, P. B.; GAGNE, G. *Fundamentos de sistemas operacionais*. 9. ed.. ed. Rio de Janeiro: LTC, 2015. 10

STALLINGS, W. *Arquitetura e organização de computadores*. 10. ed.. ed. São Paulo: Pearson, 2017. 7, 10

Anexos

ANEXO A – CÓDIGO-FONTE DO INFOGRÁFICO

A.1 REPOSITÓRIO COMPLETO

O código-fonte completo do infográfico, com histórico de *commits* e README de instruções, encontra-se em:

<<https://github.com/glkwolff/Projeto-Estruturas-de-computadores>>

A reprodução integral neste anexo é impraticável: o arquivo `projeto.html` ultrapassa 2 500 linhas e o critério de entrega prevê justamente que, em código longo, se descreva o link no relatório. As listagens a seguir trazem apenas trechos críticos ilustrativos.

A.2 ESTRUTURA DO ARQUIVO ÚNICO

```
1 <!DOCTYPE html>
2 <html lang="pt-BR">
3 <head>
4   <meta charset="UTF-8">
5   <title>Arquitetura de Servidores - Infográfico Interativo</title>
6   <style>
7     /* ~600 linhas de CSS:
8        custom properties, layout flex/grid, tabelas, animações */
9   </style>
10 </head>
11 <body>
12   <header> ... </header>
13   <div class="container">
14     <nav> <!-- 7 botões de navegação --> </nav>
15     <main>
16       <section id="mod1" class="module active"> <!-- Módulo 1 --> </
17       section>
18       <section id="mod2" class="module"> <!-- Módulo 2 --> </
19       section>
20       <section id="mod3" class="module"> <!-- Módulo 3 --> </
21       section>
22       <section id="mod4" class="module"> <!-- Módulo 4 --> </
23       section>
```

```

20     <section id="mod5" class="module">           <!-- Módulo 5 --> </
      section>
21     <section id="mod6" class="module">           <!-- Módulo 6 --> </
      section>
22     <section id="modRef" class="module">         <!-- Referências --> </
      section>
23     </main>
24 </div>
25 <footer> ... </footer>
26 <script>
27     /* ~600 linhas de JavaScript:
28        navegação, sub-abas, simulações por módulo */
29 </script>
30 </body>
31 </html>

```

Listing A.1 – Esqueleto geral do projeto.html

A.3 FUNÇÃO DE NAVEGAÇÃO POR SUB-ABAS

```

1 function showPanel(modId, panelId, element) {
2     const root = document.getElementById(modId);
3     if (!root) return;
4     root.querySelectorAll('.mod-panel').forEach(p =>
5         p.classList.remove('active'));
6     root.querySelectorAll('.mod-tab').forEach(t =>
7         t.classList.remove('active'));
8     const panel = document.getElementById(panelId);
9     if (panel) panel.classList.add('active');
10    if (element) element.classList.add('active');
11 }

```

Listing A.2 – Implementação das sub-abas dentro de cada módulo

A.4 CÁLCULO DO SIMULADOR RAID

```

1 function calcRaid() {
2     const level = parseInt(document.getElementById('rc-level').value);
3     let n = parseInt(document.getElementById('rc-disks').value);
4     const cap = parseFloat(document.getElementById('rc-cap').value);
5     const iops = parseInt(document.getElementById('rc-iops').value);
6
7     const mins = { 0: 2, 1: 2, 5: 3, 6: 4, 10: 4 };
8     if (n < mins[level]) n = mins[level];

```

```

9      if (level === 10 && n % 2 !== 0) n -= 1;
10
11      let useful, tolerable, penalty;
12      switch (level) {
13          case 0:    useful = n * cap;          tolerable = 0;          penalty = 1;
14          break;
15          case 1:    useful = cap;              tolerable = n-1;        penalty = 2;
16          break;
17          case 5:    useful = (n-1) * cap;      tolerable = 1;          penalty = 4;
18          break;
19          case 6:    useful = (n-2) * cap;      tolerable = 2;          penalty = 6;
20          break;
21          case 10:   useful = (n/2) * cap;      tolerable = n/2;        penalty = 2;
22          break;
23      }
24      const iopsR = n * iops;
25      const iopsW = Math.round((n * iops) / penalty);
26      /* ... atualização da DOM omitida ... */
27  }

```

Listing A.3 – Calculadora de IOPS e capacidade útil

A.5 APLICAÇÃO DA LEI DE AMDAHL

```

1  function updateAmdahl() {
2      const P = parseFloat(document.getElementById('am-p').value);
3      const N = parseInt  (document.getElementById('am-n').value);
4
5      const speedup = 1 / ((1 - P) + (P / N));
6      const ceiling = P === 1 ? Infinity : 1 / (1 - P);
7      const util     = ceiling === Infinity ? '--'
8                      : ((speedup / ceiling) * 100).toFixed(0) + '%';
9
10     document.getElementById('am-out-speedup').innerText = speedup.
11     toFixed(2) + 'x';
12     document.getElementById('am-out-cap').innerText     = ceiling ===
13     Infinity
14     ? 'inf'
15     : ceiling.
16     toFixed(2) + 'x';
17     document.getElementById('am-out-util').innerText     = util;
18 }

```

Listing A.4 – Simulador interativo da Lei de Amdahl

A.6 SIMULADOR DE *CACHE HIT/MISS*

```
1 function simulateCacheAccess() {
2   const r = Math.random();
3   let hitAt, cycles;
4   if (r < 0.92) { hitAt = 'l1'; cycles = 4; }
5   else if (r < 0.98) { hitAt = 'l2'; cycles = 12; }
6   else if (r < 0.995) { hitAt = 'l3'; cycles = 38; }
7   else { hitAt = 'ram'; cycles = 250; }
8
9   const order = ['l1', 'l2', 'l3', 'ram'];
10  const hitIndex = order.indexOf(hitAt);
11  order.forEach((lvl, i) => {
12    setTimeout(() => {
13      const el = document.querySelector(
14        `#cache-trace .cache-level[data-cl="${lvl}"]`);
15      if (i < hitIndex) el.classList.add('miss');
16      if (i === hitIndex) el.classList.add('hit');
17    }, i * 250);
18  });
19 }
```

Listing A.5 – Animação de travessia na hierarquia de memória

A.7 NAVEGAÇÃO PARA REFERÊNCIA CITADA

```
1 function goToRef(n) {
2   showModule('modRef', null);
3   setTimeout(() => {
4     const target = document.getElementById('ref-' + n);
5     if (!target) return;
6     target.scrollIntoView({ behavior: 'smooth', block: 'center' });
7     target.classList.add('highlight');
8     setTimeout(() => target.classList.remove('highlight'), 2500);
9   }, 200);
10 }
```

Listing A.6 – Função que vincula cita inline ao item bibliográfico

A.8 COMO EXECUTAR LOCALMENTE

1. Clonar o repositório:

```
git clone https://github.com/glkwolff/Projeto-Estruturas-de-computadores
```

2. Abrir o arquivo `projeto.html` em qualquer navegador moderno (Chrome 130+, Firefox 130+, Edge 130+).
3. Nenhuma instalação, servidor local ou conexão de internet é necessária.